PRAKTIKUM SISTEM OPERASI

MODUL 5

(Eksplorasi Proses Xinu)

Oleh:

(Faishal Arif Setiawan) (2311104066)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL/TP

1. **Soal 1**

   Jawablah pertanyaan berikut ini:
   a. Berapa banyaknya maksimum proses yang ada pada Xinu?
   b. Berapa maksimal panjang nama suatu proses pada Xinu?
   c. Berapa nilai prioritas awal pada saat proses dibuat?
   d. Ada berapa total state pada Xinu? Sebutkan!

   Jawaban:

   a. ada 50 proses yang bisa berjalan

   b. panjang maksimal nama proses adalah 16 karakter

   c.user yang menentukan sendiri nilai prioritas saat membuat proses

   d.terdapat 7 proses:

   1. PR_FREE → slot proses kosong
   2. PR_CURR → proses sedang berjalan
   3. PR_READY → siap dijalankan (di ready queue)
   4. PR_RECV → menunggu menerima pesan
   5. PR_SLEEP → sedang tidur (delay)
   6. PR_SUSP → ditangguhkan (suspend)
   7. PR_WAIT → menunggu semaphore

2. **Soal 2**

   Perintah ps adalah perintah untuk menampilkan statistik process yang berjalan. Source code dari ps tersimpan pada file xsh_ps.c. Carilah file tersebut dan beri komentar pada 20 baris terakhir di source code tersebut!

Jawaban:

```
  GNU nano 2.2.6            File: shell/xsh_ps.c                      Modified

        printf("%3s %-16s %5s %4s %4s %10s %-10s %10s\n",
                   "---", "----------------", "-----", "----", "----",
                   "----------", "----------", "----------");

        /* Output information for each process */  //mulai menampilkan data tia$

        for (i = 0; i < NPROC; i++) {  //loop untuk seluruh proses dalam tabel $
                prptr = &proctab[i]; //mengambil pointer ke proses ke-i
                if (prptr->prstate == PR_FREE) { //cek apakah slot proses tidak$
                        continue; //jika tidak digunakan,lewati ke iterasi beri$
                }
                printf("%3d %-16s %s %4d %4d 0x%08X 0x%08X %8d\n",
                        i, prptr->prname, pstate[(int)prptr->prstate],
                        prptr->prprio, prptr->prparent, prptr->prstkbase,
                        prptr->prstkptr, prptr->prstklen); //menampilkan inform$
        }
        return 0; // mengembalikan nilai 0 yang menandakan fungsi berhasil di j$
}

^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page ^U UnCut Text^T To Spell
```

```
  GNU nano 2.2.6            File: shell/xsh_ps.c                      Modified

                fprintf(stderr, "Try '%s --help' for more information\n",
                                args[0]);
                return 1;
        }

        /* Print header for items from the process table */

        printf("%3s %-16s %5s %4s %4s %10s %-10s %10s\n",
                   "Pid", "Name", "State", "Prio", "Ppid", "Stack Base",
                   "Stack Ptr", "Stack Size");  //menampilkan header kolom

        printf("%3s %-16s %5s %4s %4s %10s %-10s %10s\n",
                   "---", "----------------", "-----", "----", "----",
                   "----------", "----------", "----------");

        /* Output information for each process */  //mulai menampilkan data tia$

        for (i = 0; i < NPROC; i++) {  //loop untuk seluruh proses dalam tabel $
                prptr = &proctab[i]; //mengambil pointer ke proses ke-i
                if (prptr->prstate == PR_FREE) { //cek apakah slot proses tidak$

^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page ^U UnCut Text^T To Spell
```

Komentar yang ditambahkan pada bagian akhir xsh_ps.c menjelaskan bahwa kode tersebut berfungsi untuk menampilkan informasi semua proses yang ada di sistem Xinu. Program terlebih dahulu mencetak header sebagai judul kolom (seperti PID, nama, state, prioritas, dan informasi stack), kemudian melakukan perulangan untuk mengakses setiap entri dalam tabel proses (proctab). Pada setiap iterasi, program

mengambil data proses dan mengecek apakah proses tersebut berada pada state PR_FREE (tidak digunakan); jika iya, maka proses tersebut dilewati. Jika tidak, informasi penting seperti ID proses, nama, status, prioritas, parent process, serta detail stack akan ditampilkan menggunakan printf. Setelah semua proses ditampilkan, fungsi diakhiri dengan return 0 yang menandakan bahwa eksekusi perintah ps berhasil dilakukan.

3. **SOAL 3**

Ubahlah perintah ps (source code: xsh_ps.c) pada Xinu sehingga menampilkan informasi tambahan berupa kolom yang berisi total message yang ada pada proses seperti gambar di bawah ini:

```
xsh $ ps
Pid Name            State Prio Ppid Stack Base Stack Ptr  Stack Size Msg Content
--- --------------- ----- ---- ---- ---------- ---------- ---------- --- -------
  0 prnull          ready    0    0 0x005FDFFC 0x00FFFF04     8192   1       4
  1 rdsproc         susp   200    0 0x005FBFFC 0x005FBFC8    16384   0       0
  2 ipout           wait   500    0 0x005F7FFC 0x005F7F40     8192   0       0
  3 netin           wait   500    0 0x005F5FFC 0x005F5EE0     8192   0       0
  5 Main process    recv    20    4 0x005E3FFC 0x005E3F64    65536   0       0
  6 shell           recv    50    5 0x005D3FFC 0x005D3C7C     8192   0       0
  7 ps              curr    20    6 0x005F3FFC 0x005F3DC8     8192   0       0
```

Kolom Msg adalah banyaknya pesan yang ada dalam proses. Kolom Content adalah isi dari pesan tersebut.

Langkah pengerjaan:

• Modifikasi source code pada file xsh_ps.c

• Kompilasi ulang Xinu dengan perintah seperti pada modul sebelumnya

• Jalankan Backend VM

• Setelah sistem berjalan, jalankan perintah $ps. Pastikan hasilnya sesuai dengan contoh output pada gambar yang diberikan.

• Screenshot source kode dan output akhir hasil modifikasi

JAWABAN:

```
  GNU nano 2.2.6            File: shell/xsh_ps.c

                int msg_count;
                umsg32 msg_content;

                if (prptr->prhasmsg) {
                    msg_count = 1;
                    msg_content = prptr->prmsg;
                } else {
                    msg_count = 0;
                    msg_content = 0;
                }
                printf("%3d %-16s %s %4d %4d 0x%08X 0x%08X %8d %5d  %8d\n",
                        i, prptr->prname, pstate[(int)prptr->prstate],
                        prptr->prprio, prptr->prparent, prptr->prstkbase,
                        prptr->prstkptr, prptr->prstklen, msg_count, msg_conten$
        }
        return 0; // mengembalikan nilai 0 yang menandakan fungsi berhasil di j$
}

^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page ^U UnCut Text^T To Spell
```

```
   __    __   _____    _    _     _    _
   \ \   / /  |__ __|  | \ | |   | |  | |
    \ \/ /      | |    |  \| |   | |  | |
    / /\ \     _| |_   | \    |  | |  | |
   / /  \ \   |     |  | | \ |   \  --  /
   --    --   -----    _    _     ----
-------------------2311104066-----------------------


Welcome to Xinu!


xsh $ ps
command :1ps not found
xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFED8     8192     1      3
  1 ipout            wait   500    0 0x005FBFFC 0x005FBF30     8192     0      0
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192     0      0
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F34    65536     0      0
  5 shell            recv    50    4 0x005F7FFC 0x005F79AC     8192     0      0
  6 ps               curr    20    5 0x005F5FFC 0x005F5FC4     8192     0      0
xsh $ _
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

Perubahan pada source code xsh_ps.c dilakukan dengan menambahkan dua kolom baru, yaitu Msg dan Content, untuk menampilkan informasi pesan pada setiap proses di Xinu. Modifikasi dilakukan dengan menambahkan variabel msg_count dan msg_content di dalam loop yang mengiterasi tabel proses (proctab). Sistem kemudian mengecek atribut prhasmsg pada setiap proses; jika bernilai true, maka berarti proses memiliki pesan sehingga msg_count diisi 1 dan isi pesan diambil dari prmsg, sedangkan jika tidak maka msg_count diisi 0 dan msg_content diisi 0.

Selanjutnya, kedua nilai tersebut ditambahkan ke dalam output printf bersama informasi proses lainnya seperti PID, nama, state, prioritas, dan stack, serta header output juga diperbarui untuk menyertakan kolom baru tersebut. Dengan demikian, perintah ps kini tidak hanya menampilkan informasi dasar proses, tetapi juga status komunikasi pesan antar proses.

4. **SOAL 4**

Ubahlah perintah uptime pada Xinu sehingga menampilkan lamanya Xinu sejak booting **hanya dalam satuan menit**.

Langkah pengerjaan:

• Modifikasi source code pada file xsh_uptime.c
• Kompilasi ulang Xinu dengan perintah seperti pada modul sebelumnya
• Jalankan Backend VM
• Setelah sistem berjalan, jalankan perintah `$uptime`. Pastikan hasilnya sesuai dengan contoh output yang diinginkan
• Screenshot source kode dan output akhir hasil modifikasi

Jawaban:

```
  GNU nano 2.2.6            File: shell/xsh_uptime.c

/* xsh_uptime.c - xsh_uptime */

#include <xinu.h>
#include <stdio.h>
#include <string.h>

/*------------------------------------------------------------------------
 * xsh_uptime - shell to print the time the system has been up
 *------------------------------------------------------------------------
 */
shellcmd xsh_uptime(int nargs, char *args[])
{
        uint32 secs = clktime;

        uint32  mins = secs / 60;       /* Seconds in a minute          */

        /* For argument '--help', emit help about the 'uptime' command  */

        if (nargs == 2 && strncmp(args[1], "--help", 7) == 0) {
                printf("Use: %s\n\n", args[0]);

^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page ^U UnCut Text^T To Spell
```

```
  GNU nano 2.2.6            File: shell/xsh_uptime.c

{
        uint32 secs = clktime;

        uint32  mins = secs / 60;       /* Seconds in a minute          */

        /* For argument '--help', emit help about the 'uptime' command  */

        if (nargs == 2 && strncmp(args[1], "--help", 7) == 0) {
                printf("Use: %s\n\n", args[0]);
                printf("Description:\n");
                printf("\tDisplays time since the system booted\n");
                printf("Options:\n");
                printf("\t--help\t display this help and exit\n");
                return 0;
        }

        printf("Uptime: %d minutes\n", mins);

        /* Check for valid number of arguments */

^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page ^U UnCut Text^T To Spell
```

```
06 Apr 15:19:4btained IP address  10.0.3.15   (0x0a00030f)



----------------Faishal Arif--------------------------
   __    __   _____    _   _    _    _
   \ \  / /  |__ __|  | \ | |  | |  | |
    \ \/ /     | |    |  \| |  | |  | |
    / /\ \    _| |_   | \   |  | |  | |
   / /  \ \  |     |  | | \ |  \  --  /
   --    --   _____    _   _     ____
-------------------2311104066-----------------------


Welcome to Xinu!


xsh $ uptime
command n not found
xsh $ uptime
Uptime: 2 minutes
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
```

Modifikasi pada perintah uptime di Xinu dilakukan dengan mengubah cara perhitungan dan tampilan waktu agar hanya ditampilkan dalam satuan menit. Nilai waktu sejak sistem boot diambil dari variabel clktime yang menyimpan waktu dalam detik, kemudian dikonversi menjadi menit dengan membagi nilai tersebut dengan 60. Seluruh perhitungan lain seperti hari dan jam dihapus agar output menjadi lebih sederhana sesuai kebutuhan soal. Dalam kasus tertentu ketika timer

Xinu tidak berjalan dengan baik, nilai clktime dapat ditambahkan secara manual untuk memastikan output tidak selalu 0. Hasil akhirnya ditampilkan menggunakan printf dalam format “Uptime: X minutes”, sehingga informasi waktu menjadi lebih ringkas dan sesuai dengan spesifikasi yang diminta.